

EA4. Documentación de la Arquitectura y Modelo de Datos

Juan Esteban Ospina López

Cesar Luis Vasquez

Institución Universitaria Digital de Antioquia

Ingeniería de Software Y Datos

Infraestructura y arquitectura para Big Data

29/09/2026

1.0 INTRODUCCIÓN

Para el desarrollo de un pipeline end-to-end de Big Data. Su objetivo principal es procesar el conjunto de datos de "Cobertura móvil por tecnología" en Santander, Colombia, proveniente de la API de Datos Abiertos, transformando registros brutos en información valiosa para la toma de decisiones territoriales.

El proyecto implementa la arquitectura por capas Medallion:

- **Capa Bronze (Landing Zone):** Ingesta los datos brutos de la API web directamente en la tabla `cobertura_movil_raw` dentro de SQLite (`ingestion.db`), garantizando su trazabilidad original.
- **Capa Silver (Limpieza):** Depura la información mediante reglas de calidad de datos, corrigiendo errores tipográficos (como la normalización de `cobertuta_4g` a `cobertura_4g`), eliminando duplicados y gestionando valores nulos.
- **Capa Gold (Enriquecimiento):** Genera productos analíticos incrementales, calcula indicadores clave como `total_tecnologias_cobertura` y exporta reportes en Excel (`enriched_data.xlsx`) e informes de auditoría (`enriched_report.txt`).

El flujo está completamente orquestado y automatizado mediante un flujo de CI/CD en GitHub Actions, el cual ejecuta en secuencia los scripts de Python (`main.py`, `cleaning.py` y `enrichment.py`) y simula el funcionamiento de un Data Lakehouse empresarial en la nube.

2.0 VISION GLOBAL

El proyecto se concibe como un pipeline end-to-end de Big Data en la nube estructurado bajo una arquitectura por capas (similar al modelo Medallion con niveles Bronze, Silver y Gold). En este esquema, los datos avanzan secuencialmente a través de fases de ingesta, limpieza y enriquecimiento, incrementando gradualmente su calidad, valor analítico y estructura. Todo el flujo se encuentra orquestado de forma automática mediante integración continua (CI/CD) a través de GitHub Actions, garantizando trazabilidad, auditoría y persistencia en cada ejecución.

La primera fase corresponde a la ingesta de datos, la cual actúa como la zona de aterrizaje o *Landing Zone* (Capa Bronze). En este punto, el sistema captura e integra información proveniente de una API web llamada datos abiertos con el dataset Cobertura móvil por tecnología, Santander donde contiene 14 columnas referentes a la tecnología móvil del departamento de Santander Colombia. Todos estos registros se almacenan en su estado original y bruto dentro de la tabla de staging `cobertura_movil_raw` en SQLite. El objetivo principal de esta etapa es garantizar la ingesta completa sin modificar la estructura original ni perder trazabilidad sobre los orígenes de datos.

La segunda fase abarca el preprocesamiento y saneamiento de los datos (Capa Silver). Tomando como insumo los registros brutos almacenados en la fase anterior, el módulo de limpieza aplica reglas estrictas de calidad de datos. Durante este proceso se corrigen inconsistencias tipográficas (como la normalización de la columna `cobertuta_4g` a `cobertura_4g`), se eliminan registros duplicados, se estandarizan tipos de datos, se corrigen valores nulos y se valida la integridad territorial de municipios y centros poblados. Como resultado, los datos limpios se consolidan en la base de datos y se generan los primeros productos de salida: la hoja de cálculo `cleaned_data.xlsx` y el reporte inicial de auditoría `cleaning_report.txt`.

La tercera fase es el enriquecimiento de datos (Capa Gold), diseñada para transformar la información saneada en productos analíticos de alto valor para la toma

de decisiones e inteligencia de negocios (BI). En esta etapa se aplican transformaciones avanzadas y reglas de negocio, destacando el cálculo de métricas clave como la variable agregada `total_tecnologias_cobertura`, que evalúa la presencia de tecnologías móviles desde 2G hasta 5G por cada registro. Además, este módulo opera de forma incremental, procesando únicamente las novedades de la sesión sin sobrescribir ni eliminar el histórico almacenado. El resultado final se consolida en un reporte de auditoría completo (`enriched_report.txt`) y un libro de Excel ejecutivo (`enriched_data.xlsx`) con múltiples hojas, tablas con formato profesional y paneles de indicadores.

Finalmente, la integración global y el alojamiento en la nube se simulan mediante el motor de orquestación de GitHub Actions. El archivo de flujo de trabajo `.github/workflows/bigdata.yml` define la ejecución automatizada ante eventos de código (*push*), ejecuciones programadas o activación manual. La infraestructura aprovisiona un entorno virtual en la nube, instala las dependencias necesarias y ejecuta en orden estricto la ingesta, limpieza y enriquecimiento. Al finalizar, el pipeline valida la creación de archivos, muestra los reportes de auditoría en los logs del sistema, empaqueta las evidencias en artefactos descargables y realiza un *commit* automático para mantener sincronizado el estado final de la base de datos y reportes dentro del repositorio.

3.0 COMPONENTES PRINCIPALES

3.1. La Base de Datos Analítica (SQLite)

SQLite actúa como el motor relacional persistente y repositorio central del sistema. Su diseño ligero pero robusto permite estructurar los datos bajo un modelo por capas dentro del archivo `ingestion.db`. La base de datos alberga dos tablas principales: `cobertura_movil_raw`, destinada a almacenar la ingesta bruta sin alterar la estructura original de los orígenes heterogéneos, y `cobertura_movil_limpios`, diseñada con claves primarias autoincrementales e índices para contener la información corregida y enriquecida. ¿Un aspecto técnico clave de SQLite en este proyecto es la gestión de inserciones incrementales mediante la cláusula `SQL LIMIT -1 OFFSET ?`, la cual permite leer y procesar únicamente las novedades registradas en cada sesión,

garantizando la preservación del histórico acumulado sin riesgo de duplicación o sobreescritura.

3.2. Los Scripts de Procesamiento

Los scripts de Python constituyen el motor de transformación y lógica de negocio del pipeline, divididos funcionalmente en tres módulos:

- **Ingesta (main.py):** Es el encargado de la extracción e integración desde la API web y su función es extraer los datos en formato JSON y mediante pandas y librerías nativas del lenguaje Python se mapea cada registro a un esquema estándar de 14 campos antes de volcarlo a la capa bruta de la base de datos.
- **Limpieza (cleaning.py):** Ejecuta los controles de calidad de datos. Lee los registros brutos en SQLite para sanear imperfecciones comunes como la corrección tipográfica de columnas (por ejemplo, normalizar cobertura_4g a cobertura_4g), eliminación de espacios en blanco, estandarización de valores booleanos (S/N) y validación de códigos territoriales de municipios y centros poblados.
- **Enriquecimiento (enrichment.py):** Aplica la lógica analítica de valor agregado. Calcula indicadores clave como total_tecnologias_cobertura (sumatoria condicional de tecnologías activas de 2G a 5G), genera métricas consolidadas por operador y año, y produce las evidencias finales. Este script utiliza openpyxl para construir libros de Excel ejecutivos (enriched_data.xlsx) con múltiples pestañas, KPIs destacados y diseño visual profesional, además de redactar el informe de auditoría detallado en formato texto (enriched_report.txt).

3.3. El Mecanismo de Automatización (GitHub Actions)

La orquestación y ejecución en la nube están completamente automatizadas mediante el motor de CI/CD de GitHub Actions, configurado en el archivo `.github/workflows/bigdata.yml`. Este componente se encarga de aprovisionar de forma autónoma un contenedor efímero en Linux (ubuntu-latest), configurar el entorno de ejecución de Python 3.11 e instalar las dependencias necesarias. El workflow coordina

la secuencia de ejecución estricta (main.py → cleaning.py → enrichment.py), verifica la presencia de los archivos resultantes mediante pruebas de sistema, e imprime los reportes de auditoría directamente en los logs de la consola. Finalmente, gestiona la persistencia mediante dos vías: la publicación de evidencias como artefactos descargables y el *commit* automático de vuelta al repositorio utilizando permisos de escritura (permissions: contents: write), garantizando un ciclo de vida de integración continua sin intervención manual.

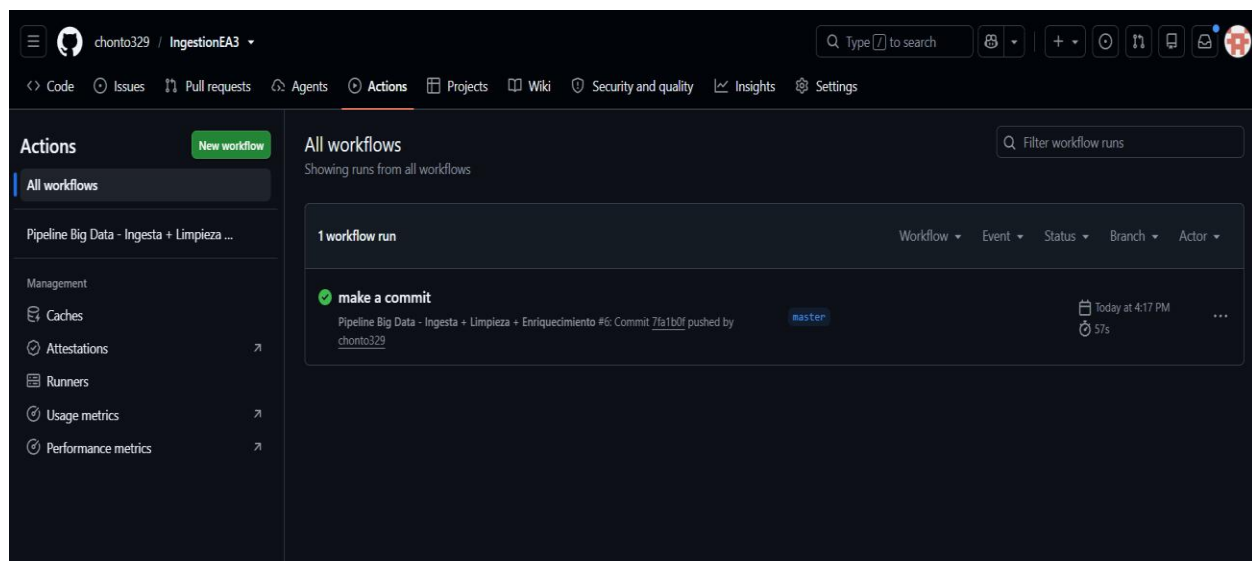


Figura 1: Pantallazo de GitHub Actions con ejecución del pipeline correctamente.

4.0 DIAGRAMAS DE ARQUITECTURA

4.1 Ingesta

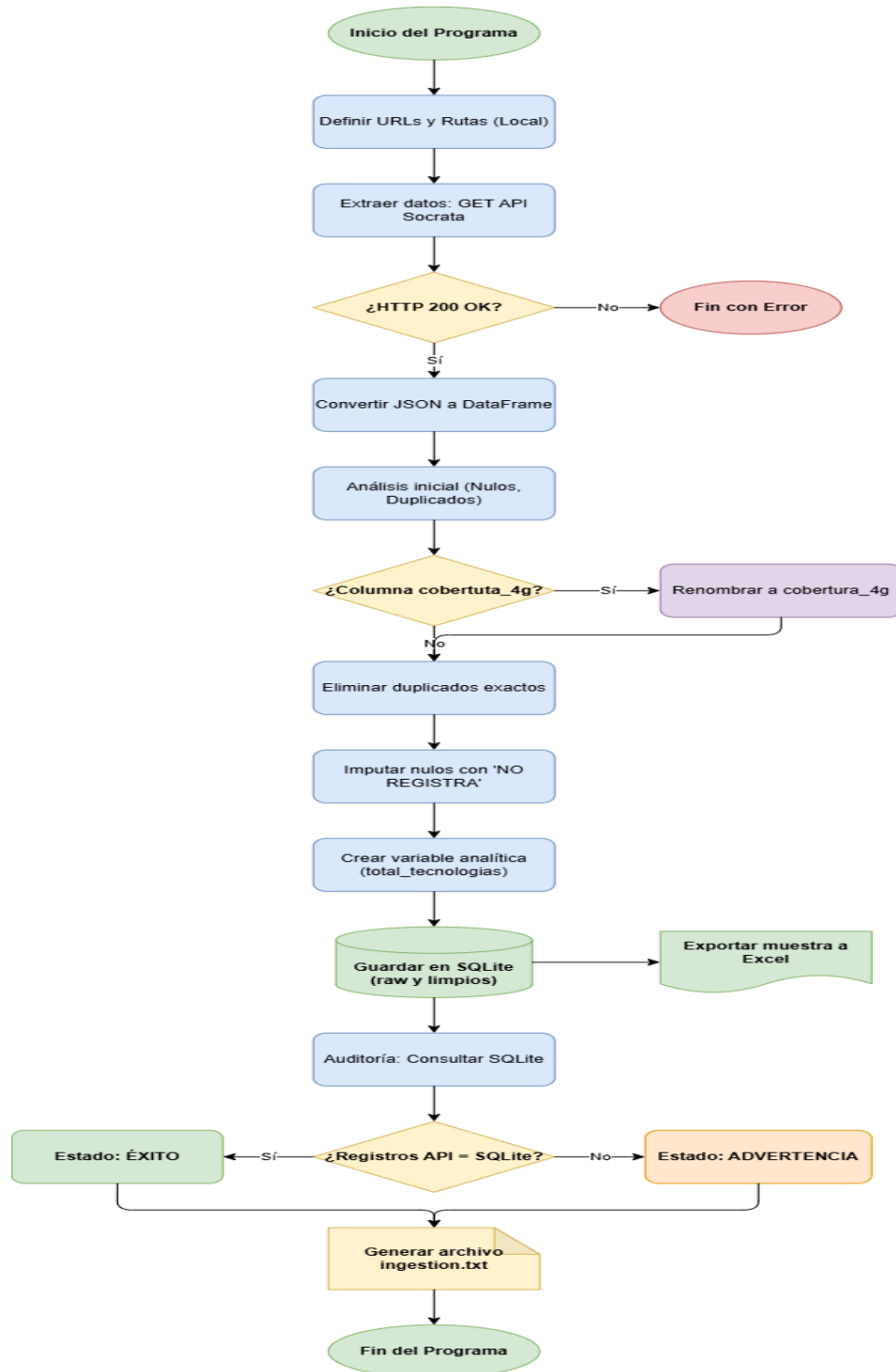


Figura 2: Diagrama de arquitectura de ingesta de datos.

4.2 Limpieza

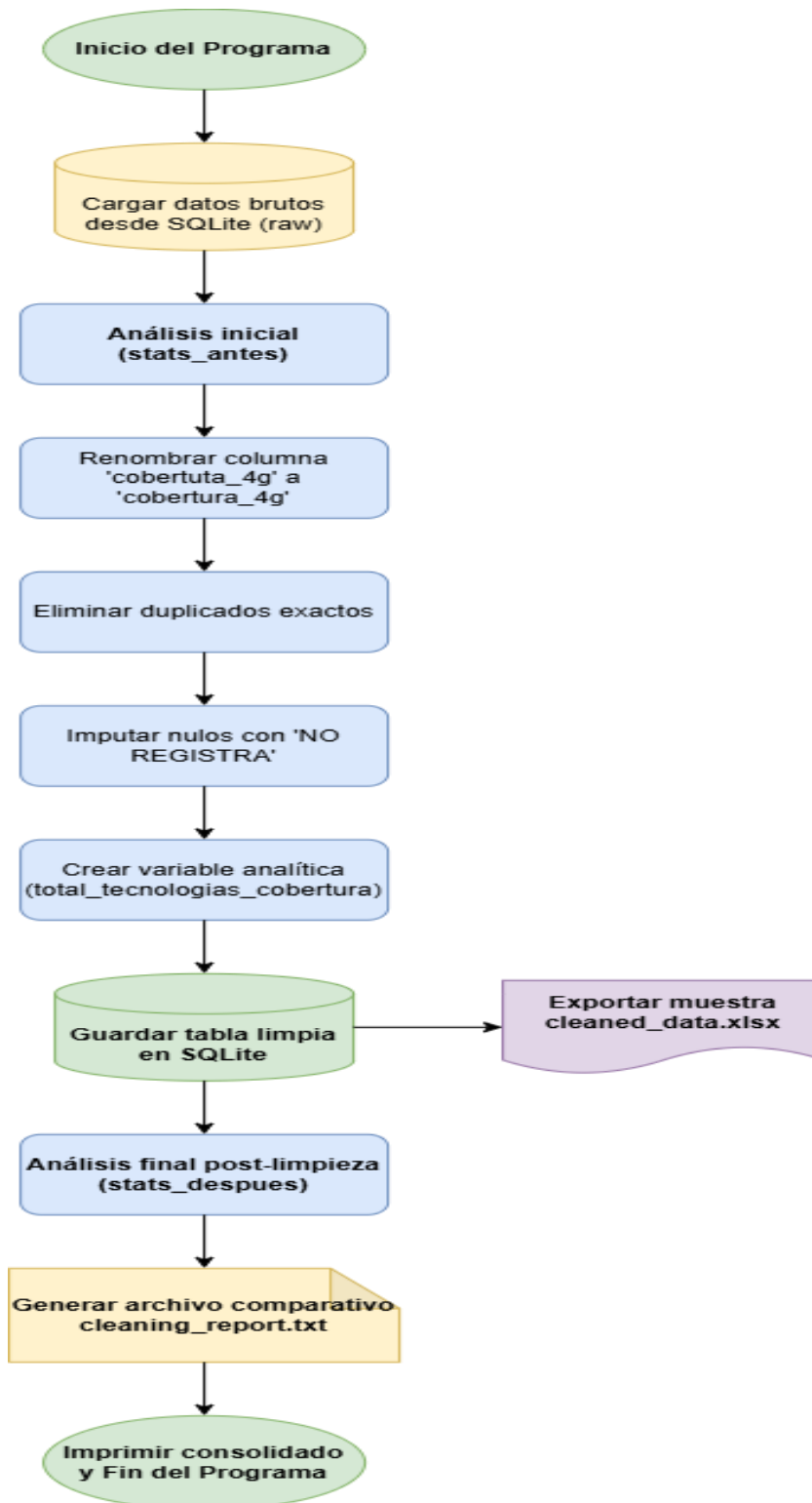


Figura 3: Diagrama de arquitectura de Limpieza de datos.

4.3 Enriquecimiento



Figura 4: Diagrama de arquitectura de enriquecimiento de datos.

5.0 MODELO DE DATOS

Tabla 1: Tabla de descripción del modelo de datos.

Nombre de la Columna	Tipo de Dato	Campo API	Descripción del Dato	Ejemplo de Valor
AÑO	Número	anio	Año correspondiente al reporte del indicador de cobertura.	2023
TRIMESTRE	Número	trimestre	Trimestre del año en que se realiza el reporte (1 a 4).	3
COD DEPARTAMENTO	Texto	cod_departamento	Código DANE oficial de dos dígitos para el departamento.	68 (Santander)
DEPARTAMENTO	Texto	departamento	Nombre oficial del departamento.	SANTANDER
COD MUNICIPIO	Texto	cod_municipio	Código DANE oficial de cinco dígitos para el municipio.	68001
MUNICIPIO	Texto	municipio	Nombre del municipio evaluado.	BUCARAMANGA

CENTRO POBLADO / CABECERA	Texto	centro_poblado	Nombre de la cabecera municipal, inspección de policía o corregimiento.	CABECERA MUNICIPAL
PROVEEDOR	Texto	proveedor	Nombre comercial del Proveedor de Redes y Servicios Móviles (PRSTM).	CLARO, TIGO, WOM, MOVISTAR
TECNOLOGÍA	Texto	tecnologia	Generación de la tecnología de red desplegada en la zona.	2G, 3G, 4G, 5G
COBERTURA / ESTADO	Texto	cobertura	Estado o presencia de señal de la tecnología reportada en la ubicación.	SI, NO, PARCIAL

5.1 Diagramas de datos

Diagrama de Modelo de Datos y Flujo de Transformación (ETL)

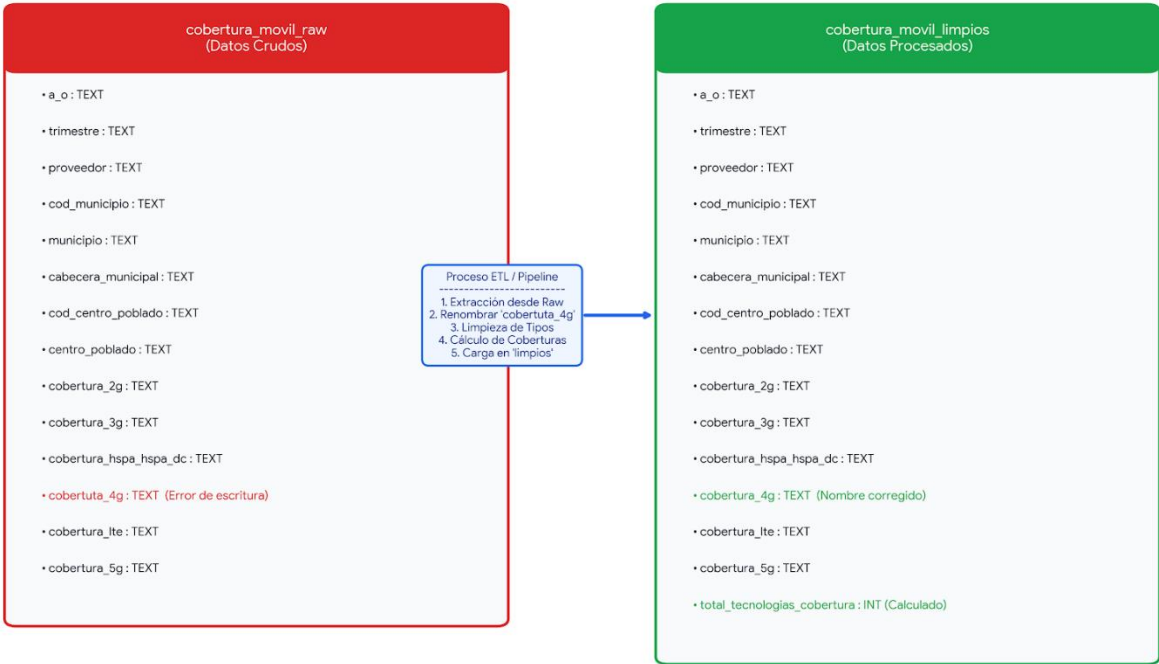


Figura 5: Diagrama de modelo de datos en SQLite.

Diagrama de Arquitectura y Flujo de Procesamiento ETL

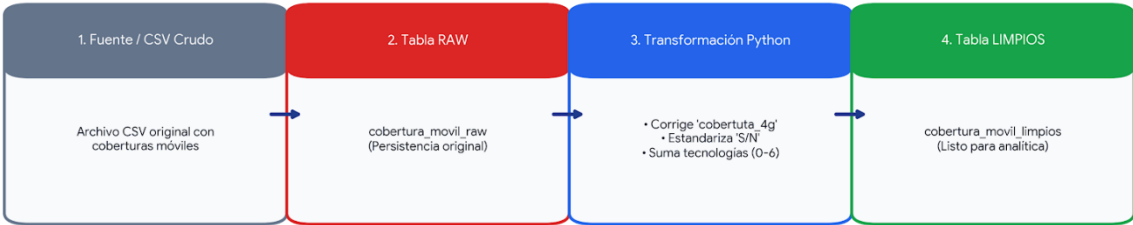


Figura 6: Diagrama de arquitectura y flujo de procesamiento ETL.

5.2 Justificación

El diseño del modelo de datos basado en dos capas principales (Capa Cruda / raw y Capa Procesada / limpios) responde a las mejores prácticas de ingeniería y analítica de datos, siguiendo una arquitectura de staging y data warehouse.

En primer lugar, la separación de responsabilidades y la trazabilidad permiten que la Capa Cruda (cobertura_movil_raw) funcione como una zona de desembarco que almacena la información exactamente como proviene de la fuente, incluyendo errores de redacción como cobertura_4g. Esto garantiza que siempre se pueda verificar la fuente original sin perder información. Además, si en el futuro cambian las reglas de negocio o de limpieza, no es necesario volver a descargar o solicitar los archivos fuente, sino que basta con volver a ejecutar el pipeline de transformación desde la tabla raw. Por su parte, la Capa Procesada (cobertura_movil_limpios) representa la única fuente de verdad para el consumo analítico, libre de anomalías de nomenclatura y estructurada para consultas óptimas.

En segundo lugar, se corrigen los errores de nomenclatura en origen. En el esquema crudo, la columna de cuarta generación presentaba un error tipográfico (cobertura_4g). Dejar este tipo de errores en la capa de consumo genera fallas en scripts automáticos, consultas SQL estandarizadas e integración con herramientas de Inteligencia de Negocios (BI). En la capa limpia, este campo se estandariza a cobertura_4g.

En tercer lugar, se realiza un enriquecimiento del dato mediante un campo calculado. Se incorpora el atributo total_tecnologias_cobertura (de tipo entero), el cual sintetiza en un único valor numérico cuántas tecnologías paralelas (2G, 3G, HSPA, 4G, LTE, 5G) ofrecen servicio en una localidad o centro poblado determinado.

Respecto a la facilidad para la integración de datos, se logran cruces geográficos y temporales simplificados. Al incluir códigos estándar (cod_municipio, cod_centro_poblado) y delimitadores temporales (a_o, trimestre), la tabla limpia se puede cruzar de forma directa mediante JOINS con otras bases de datos del proyecto, tales como censos poblacionales (DANE), capas geográficas y GIS (archivos Shapefile

/ GeoJSON para mapeo vectorial), indicadores socioeconómicos o índices de pobreza municipal, y reportes de PQR o calidad de servicio por operador. Asimismo, se facilita la interoperabilidad con herramientas de BI y ETL, ya que las herramientas de visualización como Power BI, Tableau o Looker requieren esquemas consistentes. La capa limpia proporciona una estructura homogénea que evita que los informes se rompan debido a campos mal nombrados o tipos de datos ambiguos.

En cuanto al análisis de información en etapas posteriores, este modelo permite la aceleración de consultas y reportes. El cálculo del nivel de cobertura mediante `total_tecnologias_cobertura` evita evaluar expresiones condicionales complejas (CASE WHEN) en cada consulta de análisis, lo que permite clasificar de forma inmediata áreas con brecha digital (donde el total es cero o solo cuentan con 2G) frente a zonas de alta conectividad (4G, LTE, 5G). De igual forma, facilita el análisis de evolución temporal y despliegue de red, pues la combinación de `a_o`, trimestre y proveedor permite realizar análisis de tendencias para medir la velocidad de despliegue de nuevas tecnologías por cada operador. Finalmente, impulsa la democratización del dato (Self-Service BI), dado que los analistas de negocio, economistas o tomadores de decisiones pueden consultar directamente la tabla limpia sin necesidad de realizar procesos previos de limpieza en Python, R o Excel, reduciendo el margen de error humano.

5.0 JUSTIFICACION DE HERRAMIENTAS Y TECNOLOGIAS

5.1 ELECCION DE HERRAMIENTAS

Para el desarrollo de este pipeline ETL (Extracción, Transformación y Carga), se ha seleccionado un stack tecnológico moderno que equilibra la agilidad en el desarrollo con la capacidad de escalar hacia entornos productivos. A continuación, se detalla la justificación de cada herramienta:

5.1.1 SQLite:

- **Justificación:** Es un motor de base de datos relacional ligero, autocontenido y sin servidor. En este proyecto se utilizó para consolidar la información extraída (tablas RAW y LIMPIOS).

Aporte al proyecto:

- **Eficiencia:** Al ejecutarse localmente y en un solo archivo, elimina la latencia de red durante las pruebas de ingesta y transformación.
- **Mantenibilidad:** Al no requerir configuración de usuarios, puertos o servicios externos, facilita la portabilidad del proyecto (cualquier desarrollador puede clonar el repositorio y ejecutarlo inmediatamente).

5.1.2 Pandas:

- **Justificación:** Es la biblioteca estándar de Python para el análisis y manipulación de datos estructurados en memoria.

Aporte al proyecto:

- **Eficiencia:** Su capacidad para leer nativamente múltiples formatos heterogéneos (JSON, XML, HTML, Excel, CSV) en un par de líneas de código acelera drásticamente el desarrollo.
- **Mantenibilidad:** Proporciona un código limpio y legible para las transformaciones iniciales, facilitando futuras auditorías o cambios en las reglas de negocio.

5.1.3 PySpark:

- **Justificación:** Es la interfaz de Python para Apache Spark, un motor de analítica unificada para el procesamiento de datos a gran escala (Big Data). Aunque Pandas es excelente para volúmenes pequeños/medianos, PySpark es el estándar para datos masivos.

Aporte al proyecto:

- **Escalabilidad:** Es la pieza clave para que el proyecto pueda crecer. Si en el futuro los registros de cobertura móvil pasan de miles a cientos de millones, PySpark permite distribuir el procesamiento en un clúster de múltiples nodos, algo que Pandas (limitado a un solo hilo y a la memoria RAM de una máquina) no puede hacer.

5.1.4 GitHub Actions:

- **Justificación:** Es una plataforma de Integración y Entrega Continua (CI/CD) nativa de GitHub que permite automatizar flujos de trabajo de software.

Aporte al proyecto:

- **Mantenibilidad y Eficiencia:** Permite automatizar la ejecución del pipeline. En lugar de ejecutar el script manualmente, GitHub Actions puede programarse (mediante cron jobs) para ingestar datos diariamente o ejecutarse cada vez que haya un cambio en el código, garantizando la integridad continua del flujo de datos sin intervención humana.

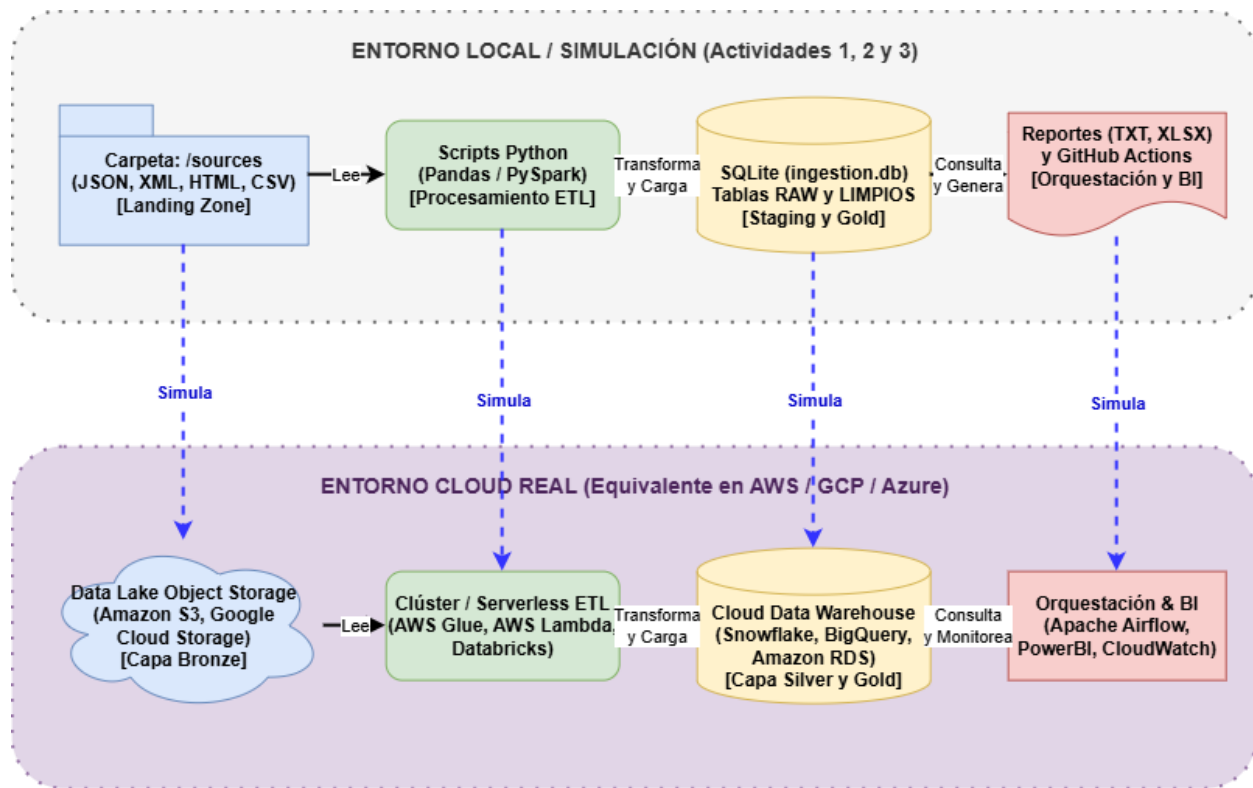


Figura 7: Diagrama de simulación de entorno real.

5.2 SIMULACIÓN DEL ENTORNO CLOUD

Aunque el proyecto se ejecutó en un entorno local (o de laboratorio) a través de las Actividades 1, 2 y 3, la arquitectura diseñada simula con exactitud el comportamiento de un Data Lakehouse moderno en la nube (AWS, Azure o GCP).

La simulación del entorno Cloud se mapea de la siguiente manera:

5.2.1 El Data Lake (Almacenamiento de Objetos en la Nube):

- En un entorno real, los archivos llegarían a un bucket de Amazon S3, Google Cloud Storage o Azure Blob Storage.
- **En nuestra simulación:** La carpeta sources/, donde se generaron y depositaron los archivos JSON, XML, Excel, CSV y HTML (Actividad 1), simula esta Capa Bronze (Raw Data) del Data Lake, actuando como zona de aterrizaje (Landing Zone) de datos no estructurados y semiestructurados.

5.2.2 El Data Warehouse / Base de Datos Relacional Cloud:

- En la nube, usaríamos servicios como Amazon RDS, Snowflake, o Google BigQuery.
- **En nuestra simulación:** La base de datos ingestion.db (SQLite) cumple este rol. Recibe los datos crudos en la tabla cobertura_movil_raw (simulando una capa Staging o Capa Silver) y, tras aplicar reglas de negocio (Actividad 2 y 3), almacena los datos depurados en cobertura_movil_limpios (simulando la Capa Gold orientada a analítica).

5.2.3 El Cómputo (Serverless o Clústeres de Procesamiento):

- En Cloud, el procesamiento se haría con AWS Glue, AWS Lambda, Databricks o Azure Data Factory.
- **En nuestra simulación:** Los scripts de Python (usando Pandas y la lógica de transformación por lotes e ingesta incremental con LIMIT y OFFSET en la Actividad 3) actúan como estos nodos de cómputo transitorios que procesan, transforman y apagan su ejecución.

5.2.4 Orquestación y Monitoreo (Auditoría):

- En la nube se utilizaría Apache Airflow, AWS CloudWatch o AWS Step Functions.
- **En nuestra simulación:** La automatización de GitHub Actions (para disparar el código) junto con la generación automática del Reporte de Auditoría (.txt) y el Dashboard en Excel (.xlsx) simulan el sistema de monitoreo, alertamiento y reportería (BI) que notifica a los Stakeholders sobre el estado de la ingesta de datos incremental.

6.0 FLUJO DE DATOS Y AUTOMATIZACION

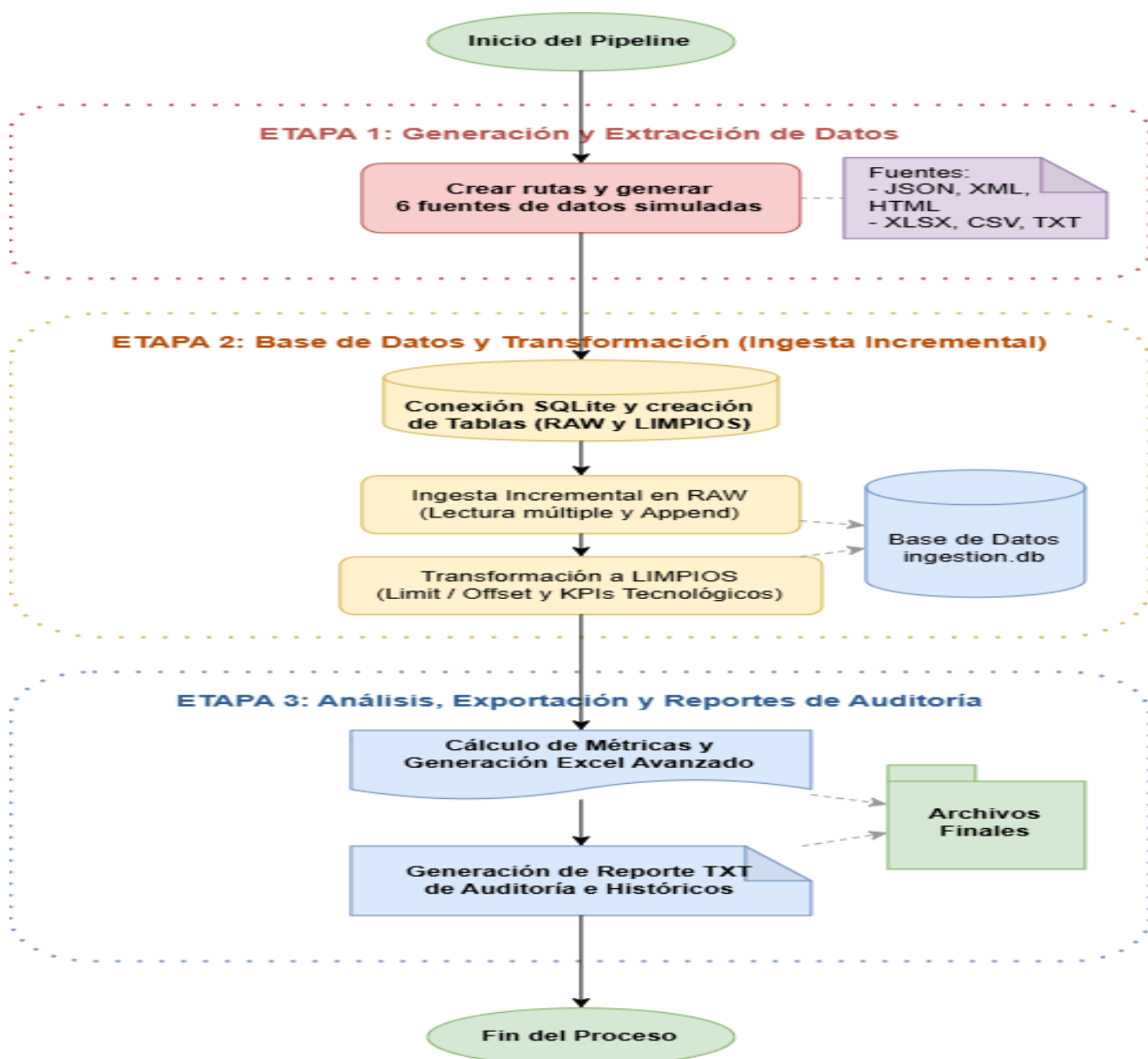


Figura 8: Diagrama de flujo de datos y automatización

7.0 CONCLUSIONES Y RECOMENDACIONES

7.1. Conclusiones: Beneficios y Limitaciones

7.1.1 Principales Beneficios de la Arquitectura y del Modelo de Datos

- **Trazabilidad y Separación de Responsabilidades:** La implementación del modelo por capas (*Medallion*: Bronze, Silver y Gold) permite preservar los datos originales sin alteraciones en la tabla cobertura_movil_raw. Esto garantiza que cualquier modificación en las reglas de negocio pueda Re ejecutarse directamente sobre la capa bruta sin necesidad de consultar nuevamente la API origen.
- **Garantía de Calidad e Integridad:** El módulo de saneamiento en la capa Silver elimina inconsistencias tipográficas (como la corrección de cobertuta_4g a cobertura_4g), elimina registros duplicados y estandariza los códigos DANE territoriales de municipios y centros poblados.
- **Valor Analítico Agregado:** La incorporación de variables calculadas como total_tecnologias_cobertura en la capa Gold sintetiza en un único valor numérico la oferta tecnológica (desde 2G hasta 5G), acelerando las consultas analíticas y la clasificación de brechas digitales sin requerir cálculos complejos en cada reporte.
- **Automatización y Trazabilidad Continua:** La orquestación mediante GitHub Actions automatiza el ciclo completo de ingesta, limpieza y enriquecimiento ante eventos en el repositorio, generando reportes de auditoría (cleaning_report.txt, enriched_report.txt) y tableros en Excel (enriched_data.xlsx) sin intervención humana.
- **Interoperabilidad Territorial:** El modelo de datos limpio facilita cruces inmediatos mediante uniones (*JOINS*) con censos poblacionales del DANE, capas geográficas (GIS/Shapefiles) e indicadores socioeconómicos.

7.1.2 Posibles Limitaciones

- **Restricciones de Escalabilidad en Almacenamiento:** El uso de SQLite como repositorio relacional (ingestion.db) es ideal para prototipado y entornos ligeros, pero presenta serias limitaciones de escritura concurrente y capacidad física ante volúmenes masivos de datos a escala empresarial.
- **Procesamiento Monomodo en Memoria:** El uso de Pandas para las transformaciones restringe el procesamiento a la memoria RAM disponible en el contenedor de ejecución (efímero en GitHub Actions), lo que puede generar cuellos de botella si el volumen de registros crece de miles a millones.
- **Dependencia de Esquemas Estáticos de la API:** Al consumir datos directamente de una API web externa, eventuales cambios de estructura o caída del servicio en el proveedor de Datos Abiertos pueden afectar la ingesta si no se cuenta con un validador estricto de esquemas.

7.2. Recomendaciones para Futuras Mejoras e Implementación en Cloud Real

- **Migración de la Zona de Aterrizaje a *Object Storage*:** Reemplazar la carpeta local por un servicio de almacenamiento masivo en la nube como Amazon S3, Google Cloud Storage o Azure Blob Storage para consolidar la Capa Bronze (*Landing Zone*) con alta disponibilidad y durabilidad.
- **Evolución a un *Cloud Data Warehouse* / *Lakehouse*:** Sustituir SQLite por motores analíticos de alto rendimiento como Snowflake, Google BigQuery, Amazon Redshift o Databricks Delta Lake para las capas Silver y Gold, permitiendo consultas distribuidas y analítica en tiempo real.
- **Escalamiento del Cómputo con PySpark / ETL *Serverless*:** Implementar plenamente PySpark en clústeres distribuidos o utilizar motores ETL *serverless*

(como AWS Glue, Azure Data Factory o Databricks) para procesar volúmenes masivos de datos sin las restricciones de memoria de Pandas.

- **Orquestación Avanzada con Monitoreo Operacional:** Evolucionar la automatización de GitHub Actions hacia un orquestador especializado en Big Data como Apache Airflow, AWS Step Functions o Prefect, permitiendo gestionar flujos con dependencias complejas, alertas automáticas, reintentos y tableros de control operacional.
- **Gobernanza de Datos y Calidad Automática:** Integrar herramientas de validación de calidad de datos en tiempo de ingesta (como *Great Expectations* o *Soda*), asegurando la detención preventiva del pipeline ante desviaciones de esquema, datos fuera de rango o registros anómalos antes de impactar las capas de consumo.